

SOLO-SMap: Semantic-Aided Online LiDAR Odometry and 3D Static Mapping for Dynamic Scenes

Ruyi Li, Shiyong Zhang*, Xuebo Zhang, Jing Yuan and Youwei Wang

Abstract—Accurate and reliable online real-time localization and mapping are crucial for autonomous navigation of robot. Dynamic objects within the perception field can affect the accuracy of registration and localization, and also introduce ghost trail artifacts in the map, hindering robot planning and decision-making. While semantic segmentation can assist in perceiving object categories, it struggles to accurately segment moving objects. In this paper, we present SOLO-SMap, a real-time localization and static map construction framework based solely on LiDAR point cloud. We leverage semantic inference to identify potential dynamic points. And then, our instance-level true dynamic points removal is achieved by utilizing geometric rules based on moving point occlusion relationships and multi-object tracking (MOT) within a nearby temporal window in the pre-alignment stage. This design preserves stable static constraints while adhering to the static world model assumption of SLAM systems, benefiting accuracy and reducing drift, particularly in busy intersections. We evaluated the performance of SOLO-SMap in dynamic scenes on KITTI datasets and our self-made datasets, and conducted a comprehensive comparison with other methods, validating the effectiveness and robustness of the proposed method. A supplementary video can be accessed at <https://www.youtube.com/watch?v=x-VKr3ag03M>.

I. INTRODUCTION

Online high-precision static map construction plays an important role in autonomous navigation of mobile robots, which can help robots achieve accurate real-time localization and optimal path planning [1]. It is well known that a 3D world map is constructed incrementally [2] by stitching together several LiDAR scans based on corresponding transformations, which is usually obtained through alignment and optimization in SLAM. For the process of alignment, the unstable dynamic points may lead to divergence of the optimizer and impair robustness [3], so a natural consideration is to use stable static keypoints [4] for pose estimation. However, in the absence of accurate relative poses, it is a challenging task to reject dynamic points accurately and quickly, which actually involves a "chicken or egg first" problem [5].

Distinguished from the multi-sensor fusion architecture, this paper explores solutions for SLAM in dynamic scenes

This work was supported in part by National Natural Science Foundation of China under Grant 62303249/62293513, in part by the Beijing-Tianjin-Hebei Fundamental Research Cooperation Project under Grant 24JCZJC00390, in part by the China Postdoctoral Science Foundation-Tianjin Joint Support Program under Grant 2023T013TJ, and in part by the China Postdoctoral Science Foundation under Grant 2024M751526.

The authors are with the Institute of Robotics and Automatic Information System, College of Artificial Intelligence, Nankai University, Tianjin 300350, China, and also with the Tianjin Key Laboratory of Intelligent Robotics, Nankai University, Tianjin 300350, China (e-mail: zhangshiyong@nankai.edu.cn; ruyi@mail.nankai.edu.cn)

* Corresponding Author

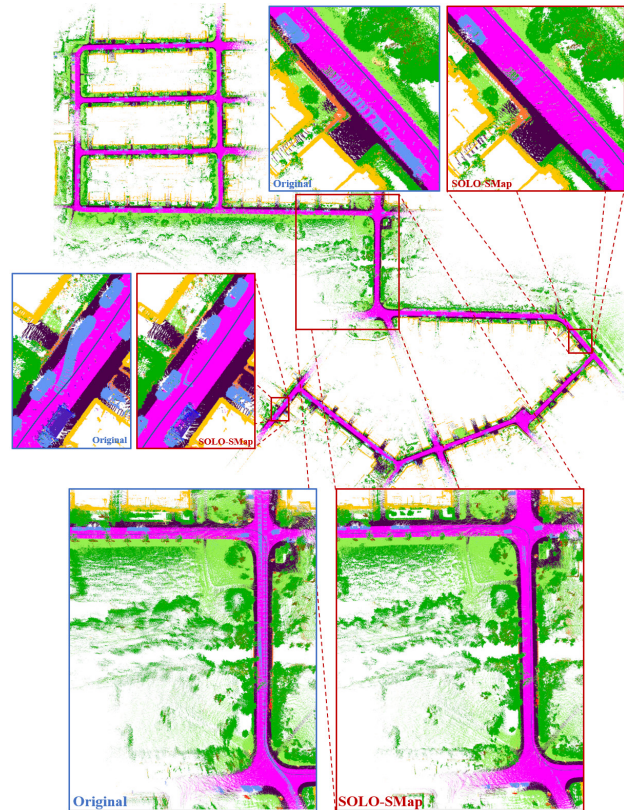


Fig. 1: Static map constructed by SOLO-SMap (outlined in red) on sequence 28 of KITTI raw dataset and comparison with map without dynamic points removal (outlined in blue).

from the perspective of sparse 3D point cloud processing, using only LiDAR for environment perception, thereby endowing the system with good portability and scalability [6].

We increase the scene tolerance of the odometry by removing dynamic points. Based on different data processing methods and time considerations, methods for removing dynamic points on maps can be divided into offline and online categories [7]. Offline methods typically consider historical time and space information or rely on prior map of the environment [8], but may not meet real-time requirements. Online methods utilize measurements of neighboring lidar scans to filter out dynamic points, but they often rely on additional SLAM that provides pose estimation [5].

When neither the poses nor the maps are available, dynamic points can be detected by segmentation-based methods. Recent advancements in deep learning have led to impressive performance from various segmentation networks,

including semantic [9], panoptic [10], instance segmentation [11] and so on. However, semantic segmentation only provides category labels, which cannot directly determine whether an object is currently in motion. It is detrimental to alignment to remove all points belonged to dynamic categories [12], as stationary objects like parked cars are valuable environment features. Furthermore, semantic inference models rely on supervised labels, and there may be misclassification. Therefore, we identify the potential dynamic points by semantic information, assuming them as very likely to be in motion, thereby narrows down the candidate scope of the true dynamic points.

This paper aims to propose an online robust simultaneous localization and mapping (SLAM) pipeline for dynamic scenes. By leveraging efficient semantic inference and carefully designed geometric rules based on occlusions of moving points along with MOT, we achieve instance-level dynamic points removal in the pre-alignment stage. It helps to prevent the *ghost trail* effect [13] in the generated map which may impair the accuracy of odometry and subsequent path planning and optimization. The performance of our system is shown as Fig. 1. The main contributions of this paper are as follows:

- 1) We propose a moving points detection strategy that mainly focuses on potential dynamic points for facilitating real-time performance.
- 2) We perceive object at instance level through potential dynamic objects tracking based on observations in the neighboring time window.
- 3) The true dynamic points are rejected in the pre-alignment stage by integrating the state estimations of potential dynamic objects and moving points detection, and we implement an online odometry for dynamic scenes and static map construction framework.
- 4) We have validated the superiority and robustness of the proposed pipeline on various datasets, including the public benchmark dataset SemanticKITTI and crowded real-world scenes.

II. RELATED WORK

Available SLAM frameworks are based on the assumption of a static world model. The early approaches did not afford distinct consideration to dynamic objects, subsuming them within anomaly detection and outlier rejection [14]. Subsequent research has commonly addressed the construction of a static map through the detection and removal of moving objects, which can be categorized into three core methodologies: ray casting-based [15], visibility-based and segmentation-based. The ray casting-based methods assume that areas traversed by light rays are free. They represent occupancy probability by the hit rate of voxels in space [16]. However, as the resolution increases, the computational complexity of these methods becomes intractable [13], especially when real-time operation is considered. The visibility-based methods are similar to ray casting-based methods, assuming the point closer along a laser beam as dynamic. By focusing on a narrow field of view, they query the

difference between scanned points and local map, and the point observed previously is considered dynamic if the query point locates behind it [17]. However, they fail to detect large obstacles when there are no points behind [18]. And both ray casting and visibility based methods struggle with ambiguity caused by large incidence angles [19]. For the last category, segmenting terrain based on plane features and posed clustering can assist in distinguishing moving objects, but its effectiveness is reduced in highly dynamic environments. In recent years, deep neural networks have been used to recognize 3D objects and perform orientation regression to predict dynamic points [20]. But they have shown poor generalization and interpretability. Therefore, research has turned to appearance-based solutions. Semantic segmentation can identify the category to which a point belongs, allowing for integration with traditional methods to remove dynamic points through motion or semantic consistency checks [21], which is the inspiration for this paper.

Laser-based SLAM frameworks typically follow a frontend-backend architecture [22], where the frontend performs registration to output high-frequency pose estimation, and the backend constructs some specific form of environment model through further optimization [2]. For the frontend, the ideal solution is to use stable static points for alignment. Currently, dynamic points rejection in the frontend is mainly achieved through traditional approaches or deep learning. The traditional methods identify dynamic points by checking scan-to-scan or scan-to-submap differences [23], or by employing statistical methods to filter outliers. Deep learning-based techniques utilize large amounts of labeled point cloud data to train segmentation models, extracting local and global features, and inferring the probability distribution of scanned points to segment dynamic objects [24]. This can provide clean points suitable for SLAM. Some work temporarily ignore the impact of dynamic objects during alignment and utilize semantic labels for consistency checking to segment dynamic voxels [25], but it is not suitable for highly dynamic scenes.

As for mapping, the presence of dynamic objects can lead to ghost-trail artifacts in the map. Some methods attempt to remove dynamic objects during mapping in the backend. To achieve real-time performance, some works only process keyframes those will be inserted into the map or discard scan-to-map comparison, considering only the differences between adjacent frames [17]. Apart from minimizing the number of point samples for checking, another idea is to improve the efficiency of the filtering process. DynamicFilter [7] roughly filters dynamic objects based on visibility in the frontend and then finely optimizes the voxel map in the backend to obtain a static submap. Inspired by this, we adopt a coarse-to-fine strategy for accurate removal of dynamic points.

III. METHODOLOGY

A. Problem Formulation & System Overview

The overall framework of SOLO-SMap is presented in Fig. 2. First of all, we conduct semantic inference on LiDAR point cloud with intensity to obtain semantic labels

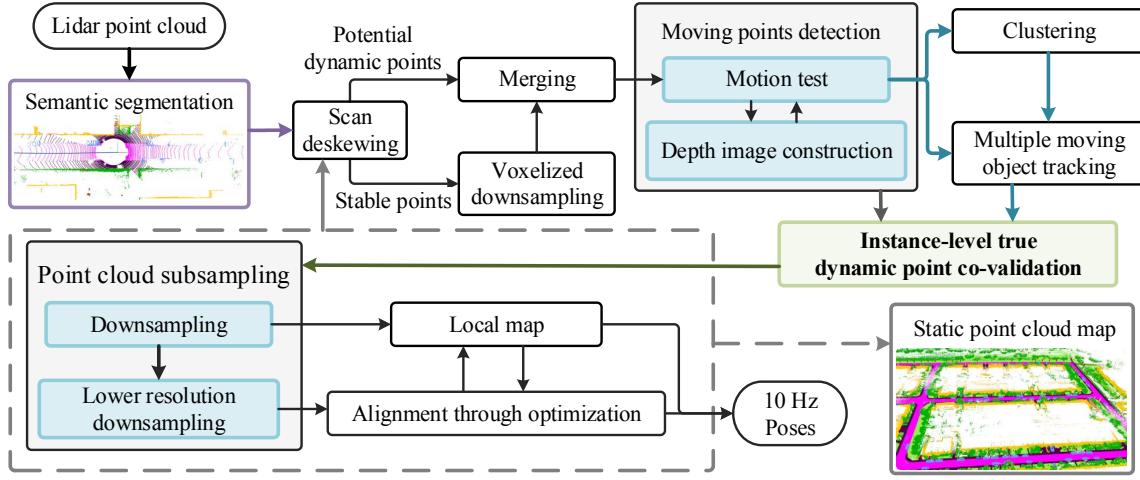


Fig. 2: Overall framework of SOLO-SMap.

for each point. Then, the constant velocity model [26] is used for motion prediction and scan deskewing, and we denote the deskewed point cloud of the k -th sweep as $\mathcal{P}_k^L = \{\mathbf{p} | \mathbf{p} \in \mathbb{R}^3\}$. Our goal is to accurately detect and remove true dynamic points in $\mathcal{P}_k^L$ in the pre-alignment stage, and utilize the remaining static points to construct an optimization problem for solving the pose of LiDAR ${}^W_L \mathbf{T}_k = [{}^W_L \mathbf{R}_k | {}^W_L \mathbf{t}_k]$ in the world coordinate system.

Specifically, the LiDAR scan $\mathcal{P}_k^L$ can be classified into potential dynamic points $\hat{\mathcal{P}}_{k,dyn}$ and stable points $\hat{\mathcal{P}}_{k,sta}$ based on semantic information. We apply voxel downsampling to $\hat{\mathcal{P}}_{k,sta}$, which mostly serve as background, in order to focus on the the potential dynamic points in subsequent moving points detection. While exploiting geometric rules designed based on occlusion relationships for moving points detection, we perform multi-object tracking on potential dynamic points based on Kalman filtering, and then combine them to segment true dynamic points $\mathcal{P}_{k,dyn}$ thus achieving true dynamic points reject at instance level. Finally, the retained static points with semantic labels $\mathcal{P}_{k,sta}$ are used for alignment and map construction. The relationships of the point sets defined in this paper are as follows:

$$\mathcal{P}_k^L = \hat{\mathcal{P}}_{k,dyn} \cup \hat{\mathcal{P}}_{k,sta} = \mathcal{P}_{k,dyn} \cup \mathcal{P}_{k,sta} \quad (1)$$

$$\hat{\mathcal{P}}_{k,dyn} = \mathcal{P}_{k,dyn} \cup \mathcal{P}_{k,dyn}^* \quad (2)$$

$$\mathcal{P}_{k,sta} = \hat{\mathcal{P}}_{k,sta} \cup \mathcal{P}_{k,dyn}^* \quad (3)$$

where $\mathcal{P}_{k,dyn}^*$ represents the points within the potential dynamic set $\hat{\mathcal{P}}_{k,dyn}$ that are actually static.

B. Semantic-based Potential Dynamic Points Segmentation

We employ RangeNet++ for semantic inference to obtain point-wise labels for each LiDAR scan. RangeNet++ projects 3D point cloud into an range image via spherical projection. It performs 2D semantic segmentation by fully convolutional neural network, and then reconstructs the point cloud on GPU for post-processing. For brevity, refer to [9] for specific details. We aim to detect the true dynamic points that

interfere with registration. The vast majority of points act as reference and background points during detection, making individual examination redundant. Thus, aimed with semantic information, we focus on the points from the LiDAR scan that belong to objects that may potentially move (such as car, bicycle, truck, motorcycle and person), considering them as potential dynamic points $\hat{\mathcal{P}}_{k,dyn}$, while temporarily treating other points as stable points $\hat{\mathcal{P}}_{k,sta}$.

C. Moving Points Detection

The principle of moving points detection based on the occlusion relation is inspired from [27]. We project the received points to depth image for moving detection. Different from [27], our framework does not have an additional odometry to provide pose estimation. Therefore, we use constant velocity model to predict a coarse priori pose for the current frame, denoted as ${}^W_L \hat{\mathbf{T}}_k = [{}^W_L \hat{\mathbf{R}}_k | {}^W_L \hat{\mathbf{t}}_k]$ for the k -th scan. To construct the depth image, for a global point ${}^W \mathbf{p}$, we first need to transform it into the target LiDAR coordinate system:

$${}^L \mathbf{p} = \mathbf{R}^{-1}({}^W \mathbf{p} - \mathbf{t}) \quad (4)$$

where ${}^L \mathbf{p} = [{}^L p_x, {}^L p_y, {}^L p_z]$ denotes the coordinate of the point in the depth image frame. For $\mathbf{p} \in \mathcal{P}_k^L$, ${}^W \mathbf{p} = {}^W_L \hat{\mathbf{R}}_k \mathbf{p} + {}^W_L \hat{\mathbf{t}}_k$, $\mathbf{R}$ and $\mathbf{t}$ correspond to the previously estimated pose of LiDAR ${}^W_L \mathbf{R}_{k-n}$ and ${}^W_L \mathbf{t}_{k-n}$ ($n > 0$). We can then get the spherical coordinates from the transformed point:

$$\varphi = \arctan 2({}^L p_y, {}^L p_x) \quad (5)$$

$$\theta = \arctan 2\left(\sqrt{{}^L p_x^2 + {}^L p_y^2}, {}^L p_z\right) \quad (6)$$

$$d = \sqrt{{}^L p_x^2 + {}^L p_y^2 + {}^L p_z^2} \quad (7)$$

And then the point is indexed in the 2D pixel plane:

$$i = \lfloor (\varphi + \pi)/r_h \rfloor, \quad j = \lfloor (\theta + \pi/2)/r_v \rfloor \quad (8)$$

where r_h and r_v are the horizontal and vertical resolution of the depth image, and $\lfloor \cdot \rfloor$ denotes the floor function.

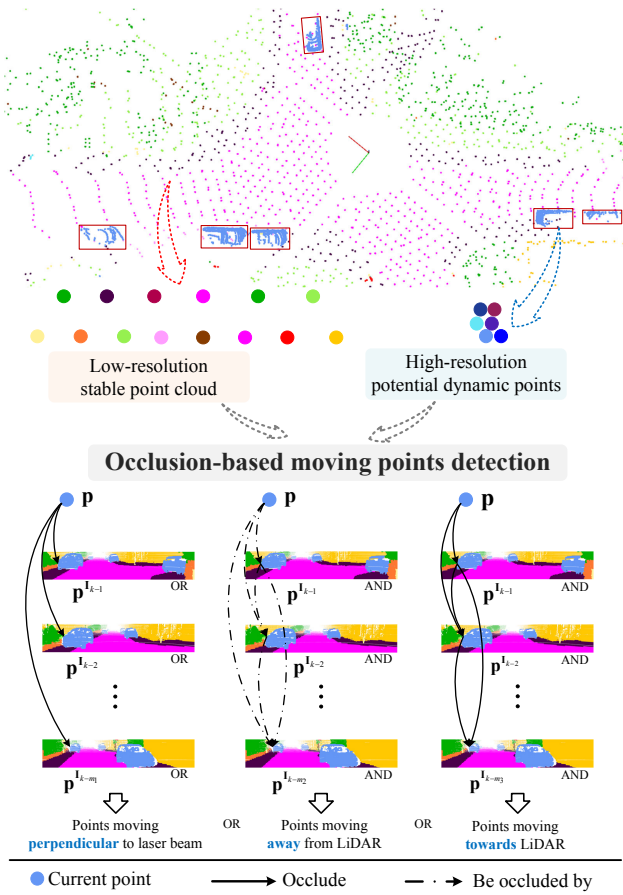


Fig. 3: Occlusion-bases moving points detection mainly focus on potential dynamic points.

Our moving detection mainly focuses on the potential dynamic points. So we apply voxel downsampling to the stable points $\hat{P}_{k,sta}$ to get $\hat{P}_{k,sta}^\#$. The depth image I_k is formed by $\hat{P}_{k,det} = \hat{P}_{k,dyn} \cup \hat{P}_{k,sta}^\#$. Subsequently, we apply occlusion based moving point detection on depth image. If the depth of the current point $\mathbf{p}$ is smaller than the minimum depth (with threshold ε_d) in a neighborhood of its pixel, it is considered to occlude points in the depth image. Otherwise, if larger than the max depth (with threshold ε_d), it is considered occluded. The three detection principles are described as: i) Detection for points of objects moving perpendicularly to the laser beam. Objects moving in this manner will occlude previously observed background. If $\mathbf{p}$ occludes more than m_1 previous depth images, then the point is considered positive (i.e., a moving point). ii) Detection for points of objects moving away from LiDAR in parallel to the laser beam. We check whether $\mathbf{p}$ is occluded by points in the previous m_2 depth images, and examine if each of these points is occluded in its previous depth images. iii) Detection for points of objects moving towards LiDAR in parallel to the laser beam. We check whether $\mathbf{p}$ occlude any points in the previous m_3 depth images, and examine if each of these points occlude its previous points in depth images. If any of the above three principles is detected as positive, the point

is considered a moving point.

D. Instance-level True Dynamic Points Removal

In III-C, we detect motion at point level. Relying solely on the moving points detection cannot completely remove moving objects, especially when the point cloud is sparse. So in this section, with the assistance of semantic labels, we perform clustering and multi-object tracking of potential dynamic objects. By combining with the moving points detection, we achieve accurate segmentation and removal of moving instances.

Clustering is performed on $\hat{P}_{k,dyn}$ by applying DBSCAN to the voxels containing potential dynamic points. It reduces computational cost compared to clustering individual points. Moreover, the clustering filters out isolated points which are often introduced by semantic segmentation errors, enhancing the accuracy of our dynamic object detection. Assume that there are n_k objects detected, the moving possibility of each potential dynamic object Pr_k^i can be represented as:

$$Pr_{(k,i)} = |\mathcal{P}_{k,mov}^i| / |\hat{P}_{k,dyn}^i| \quad (9)$$

where $i \in \{1, 2, \dots, n_k\}$. $\mathcal{P}_{k,mov}^i$ denotes the moving points on object i that is detected by the process in Sec.III-C.

The Tracked objects are initialized based on the clustering results, encompassing attributes such as orientation, velocity, centroid, and bounding box dimensions. Kalman filter is employed as the probabilistic inference model for predicting and updating the state of objects:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{u}_k + \mathbf{w}_k \quad (10)$$

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k \quad (11)$$

where $\mathbf{w}_k \sim N(0, \mathbf{R})$, $\mathbf{v}_k \sim N(0, \mathbf{Q})$. The state of an object is represented by a six-dimensional vector:

$$\mathbf{x} = [x, y, z, v_x, v_y, v_z] \quad (12)$$

Assume that there are m tracks of objects in $\hat{P}_{k-1,dyn}$ and l candidate detections in $\hat{P}_{k,dyn}$, defined respectively as:

$$X_{k-1} = \{\mathbf{x}_{k-1}^1, \mathbf{x}_{k-1}^2, \dots, \mathbf{x}_{k-1}^m\} \quad (13)$$

$$O_k = \{\mathbf{o}_k^1, \mathbf{o}_k^2, \dots, \mathbf{o}_k^l\} \quad (14)$$

The 3D multi-object tracking problem involves determining a sequence of the posteriori estimation of the set of tracked states, represented as $X_0, X_1, \dots, X_k$.

A constant velocity model is used to predict the prior state estimate $\tilde{\mathbf{x}}_k$ for each object. The Hungarian algorithm then performs the matching between O_k and $\tilde{X}_k$ to associate candidate detections with tracked objects. For achieving optimal matching, scores in the association matrix are weighted based on differences in centroid location, orientation, bounding box size, point cloud quantity, and appearance features. And then the Kalman filter is updated. The association score acts as a weighting factor, with greater similarity leading to a stronger update. Finally, we update the state of objects (tracked or not) using the optimal velocity estimation. We co-validate

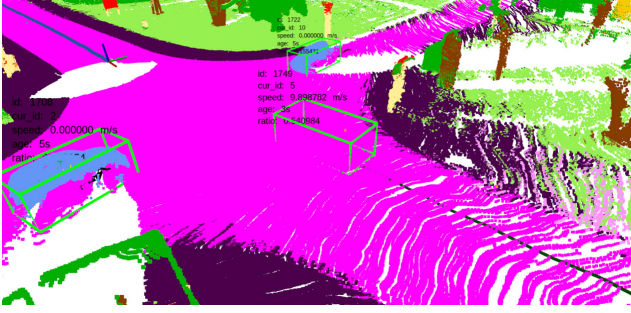


Fig. 4: Tracking and co-validation result at intersection.

the result of geometry detection and the tracked velocity to acquire an "dynamic probability" of instance:

$$D_{(k,i),label} = \frac{Pr_{(k,i)} + \log(1 + \sqrt{v_x^2 + v_y^2 + v_z^2})}{1 + e} \quad (15)$$

where *label* indicates the semantic category. An instance is considered as dynamic when $D_{(k,i),label} > \tau_{label}$. Furthermore, if the displacement of an object within a previous time window exceeds threshold d_{th} , it will also be identified as a dynamic instance. This helps in recognizing objects that have just stopped. The points belong to the true dynamic instances are rejected for facilitating subsequent alignment and mapping.

E. Alignment and Robust Optimization

The odometry of SOLO-SMap is implemented based on the fast and robust point-to-point ICP proposed in [6]. Upon completion of the aforementioned procedures, we leverage clean semantic-labeled static point cloud $\mathcal{P}_{k,sta}$ for data association and optimization.

Instead of downsampling the input point cloud by extracting keypoints (normally planar and edge feature points), we employ voxel downsampling for enhancing the tolerance of different point cloud resolution and parameters. We adopt a subsampling strategy [6]: initially downsampling $\mathcal{P}_{k,sta}$ with resolution αv ($\alpha \in (0.0, 1.0]$) to get an intermediate point cloud $\mathcal{P}'_{k,sta}$ for updating the local map, followed by downsampling $\mathcal{P}'_{k,sta}$ with a lower resolution βv ($\beta \in [1.0, 2.0]$), which results in a further reduced point cloud $\mathcal{P}''_{k,sta}$ for efficient registration, where v is the voxel grid size of local map.

The data association relies on the nearest neighbor principle. Ideally, two associated points should share the same semantic label. However, in practice, inaccurate pose predictions and imperfect semantic segmentation limit the effectiveness of semantic information for registration. This is especially challenging in noisy scenarios such as intersections, where there are significant nonlinear motion and dynamic objects, making it difficult to completely remove dynamic objects from the original point cloud. Considering these factors, we define the semantic Euclidean distance (SED) between the source point $\mathbf{p}_s$ and a point $\mathbf{p}_{s,j}$ in its neighborhood as follows:

$$d_{se,j} = \gamma \|\mathbf{p}_s - \mathbf{p}_{s,j}\|_2 \quad (16)$$

where the coefficient γ is calculated as:

$$\gamma = \begin{cases} e^{-\mu N_s} & \text{if } l(\mathbf{p}_s) = l(\mathbf{p}_{s,j}) \text{ or } l(\mathbf{p}_s)l(\mathbf{p}_{s,j}) = 0 \\ 1 & \text{otherwise} \end{cases} \quad (17)$$

where $l(\cdot)$ denotes the semantic label, and $l(\cdot) = 0$ indicates that the point is unlabeled. N_s represents the number of points within the neighborhood sharing the same semantic label as $\mathbf{p}_s$, and the constant factor μ (set to 0.1) governs how much the semantic information of neighboring points affects the calculated semantic distance. A larger N_s implies a greater likelihood of successful association with points of the same semantic label. We consider points with SED larger than τ_k as outliers and store the target point with the smallest SED. $\tau_k = 3\sigma_k$ is the adaptive threshold calculated by analyzing the deviation between the odometry and the predicted poses:

$$\sigma_k = \sqrt{\frac{1}{|\mathcal{M}_k|} \sum_{i \in \mathcal{M}_k} \delta(\Delta \mathbf{T}_i)^2} \quad (18)$$

$$\mathcal{M}_k = \{i | i < k \wedge \delta(\Delta \mathbf{T}_i) > \delta_{min}\} \quad (19)$$

where $\delta(\Delta \mathbf{T}_i)$ represents the deviation between odometry and prediction before k , detailed in [6].

Finally, we apply robust optimization on minimizing point-to-point residuals and integrate the static points (the down-sampled point cloud $\mathcal{P}'_{k,sta}$) incrementally into the map.

IV. EXPERIMENTAL EVALUATION

We conducted experiments and evaluations of our method and comparative methods on a laptop equipped with an Intel i7-10870H CPU (2.20 GHz, 16 cores), 16GB of RAM, and a single GeForce RTX 3060 Ti graphics card. The semantic inference model RangeNet++ was trained on sequences 00 10 of the SemanticKITTI dataset, which is able to run online while performing localization and mapping.

A. Odometry evaluation

The experiment is designed to validate that our proposed method SOLO-SMap is capable of estimating the ego-pose of the vehicle online across various dynamic scenarios, thereby enabling the generation of consistent environmental model. To this end, we select 10 sequences from KITTI raw data that feature high numbers of dynamic objects, encompassing city, residential and road scenes. The trajectory lengths of these sequences range from 255 to 4208 meters. These test sequences are distinct from those used in the KITTI odometry benchmark for training semantic segmentation model, thus showcasing the certain generalization capability of our semantic inference model. In practice, the performance of deep neural networks is inevitably influenced by data quality and changes in data distribution. Consequently, we must incorporate the inference errors into subsequent processing. SOLO-SMap utilizes moving points detection and instance-level multi-object tracking, which effectively tolerates the segmentation errors. And the semantic-geometric dual checking alleviates the reliance on semantic labels. The tracking

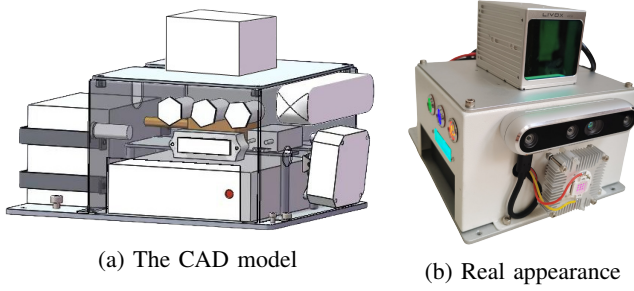


Fig. 5: Our sensor unit used in the experiment, consisting a Livox AVIA LiDAR, an Intel RealSense D455 camera, an Xsens MTi-300 IMU and an Intel NUC with Intel i7-1260P CPU. It can be mounted on various mobile robot bases.

and co-validation result is shown in Fig. 4. When the test vehicle reached the intersection, we detected three vehicles. Two of them had stopped at the intersection, while one was moving. Through co-validation, we successfully removed the moving vehicle while retaining the stationary ones.

We evaluated the absolute pose error (APE) and relative pose error (RPE) in meters for the estimated trajectories, and collected experimental results from LOAM, KISS-ICP, and SuMa++ for comparison with our proposed odometry SOLO. Suma++ is the state-of-art surfel-based LiDAR SLAM, where the semantic labels are also computed by RangeNet++. The results are shown in Tab. I. It can be seen that our method SOLO exhibits excellent performance on the test sequences of different scenarios.

Additionally, we validated SOLO on a self-recorded sequence, which was collected using a UGV (Unmanned Ground Vehicle) equipped with a sensor unit consisting of a Livox AVIA LiDAR, an Intel RealSense D455 camera, an Xsens MTi-300 IMU and an Intel NUC. The sensor unit is shown in Fig. 5. For convenience, we made the start and end of the sequence overlap. The sequence was recorded on a noisy campus road, featuring a variety of dynamic objects such as pedestrians, cyclists, cars, and delivery vans. In this experiment, we did not perform semantic segmentation on the input point clouds. Instead, we detected moving points based solely on geometric rules and, as described in Sec. III-D, tracked and co-validated moving objects to remove dynamic instances. The estimated trajectory of SOLO was projected onto a satellite image provided by Google Map, as shown in Fig. 6. The starting point and endpoint can perfectly overlap.

B. Static map assessment

In this section, we perform quantitative and qualitative analysis of constructed static maps. The point-wise assessment benchmark proposed in [19] is employed as evaluation tool. We do not apply voxel-downsampling to point cloud map due to the variation in evaluation metrics with voxel size. In addition to Static Accuracy (SA%) and Dynamic Accuracy (DA%), the composite metrics of both is also measured through Average Accuracy (AA%) and Harmonic Accuracy (HA%):

$$AA = \sqrt{SA \cdot DA} \quad (20)$$



Fig. 6: Satellite image of the campus building where the self-recorded sequence are collected. The blue line is the projection of the estimated trajectory from SOLO.

$$HA = \frac{2 \cdot SA \cdot DA}{SA + DA} \quad (21)$$

AA can better reflect poorer accuracy than HA, while HA can provide a better reflection of the balance between the two.

In order to investigate the influence of pose on dynamic points removal and accuracy of static map, we test approaches using three different pose sources. The evaluation of static map on Seq. 07 (630-820) of SemanticKITTI dataset is presented in Tab. II. The GT poses of KITTI refer to the ground truth poses obtained through a combination of differential GPS and IMU sensors in KITTI odometry benchmark. In the SemanticKITTI dataset, the estimated poses from SuMa are used as reference ground truth. In the last column, SOLO provides the poses corresponding to each LiDAR frame. It is well understood that the detection of dynamic objects relies on pose estimation. Therefore, more accurate and consistent poses contribute to more reliable segmentation results. SOLO can provide excellent pose estimates, aiding in achieving higher accuracy for static maps. Although the precision of our static map construction SMap is slightly inferior to the modified OctoMap [28], the difference is minimal, especially when considering the real-time performance and processing speed of SOLO-SMap. Moreover, SOLO-SMap exhibits a high SA, meaning it can effectively preserve static features and constraints, resulting in more accurate and robust localization.

The local map constructed by SOLO-SMap and other comparison methods are shown in Fig. 7. All of the three comparison methods are offline, which rely on additional pose estimation. Given the previous analysis, SOLO has good performance and thus we use our SOLO to provide reference poses of point cloud frames for each method. Fig. 7(a)~(e) represent the mapping results on Seq. 02 (860~950) of SemanticKITTI dataset. The result demonstrate SOLO-SMap's effectiveness in detecting and removing the points of small moving objects within the surrounding environment. The red points in the figures represent false negatives, i.e., dynamic points that were not successfully detected by the algorithm. The modified OctoMap performs best on this

TABLE I: Absolute pose error (APE) / relative pose error (RPE) on KITTI raw.

Approach	04 road	09 city	11 city	18 residential	22 residential	27 residential	28 residential	32 road	47 road	101 road	Average
LOAM	1.15/0.89	0.61/0.79	0.27/1.86	3.39/1.47	1.84/1.36	0.93/1.05	18.3/1.33	0.76/3.87	5.61/0.90	6.28/4.31	3.91/1.78
KISS-ICP	0.42/ 0.75	0.58/0.78	0.37/1.83	3.48/1.25	1.84/1.29	0.92/1.04	16.7/1.16	0.76/3.23	1.68/0.90	6.24/3.94	3.30/1.62
SuMa++	0.57/0.76	1.86/0.79	0.27/1.86	4.56/1.25	1.91/ 1.28	1.29/1.04	12.6/1.16	0.42 /3.24	6.85/0.90	3.95/3.95	3.42/1.62
SOLO	0.39 /0.76	0.55 / 0.72	0.24 / 1.83	3.17 / 1.25	1.79 /1.29	0.89 / 1.04	12.0 / 1.15	0.53/ 3.22	1.51 / 0.89	3.20 / 3.93	2.43 / 1.60

Note: APE/RPE in m.

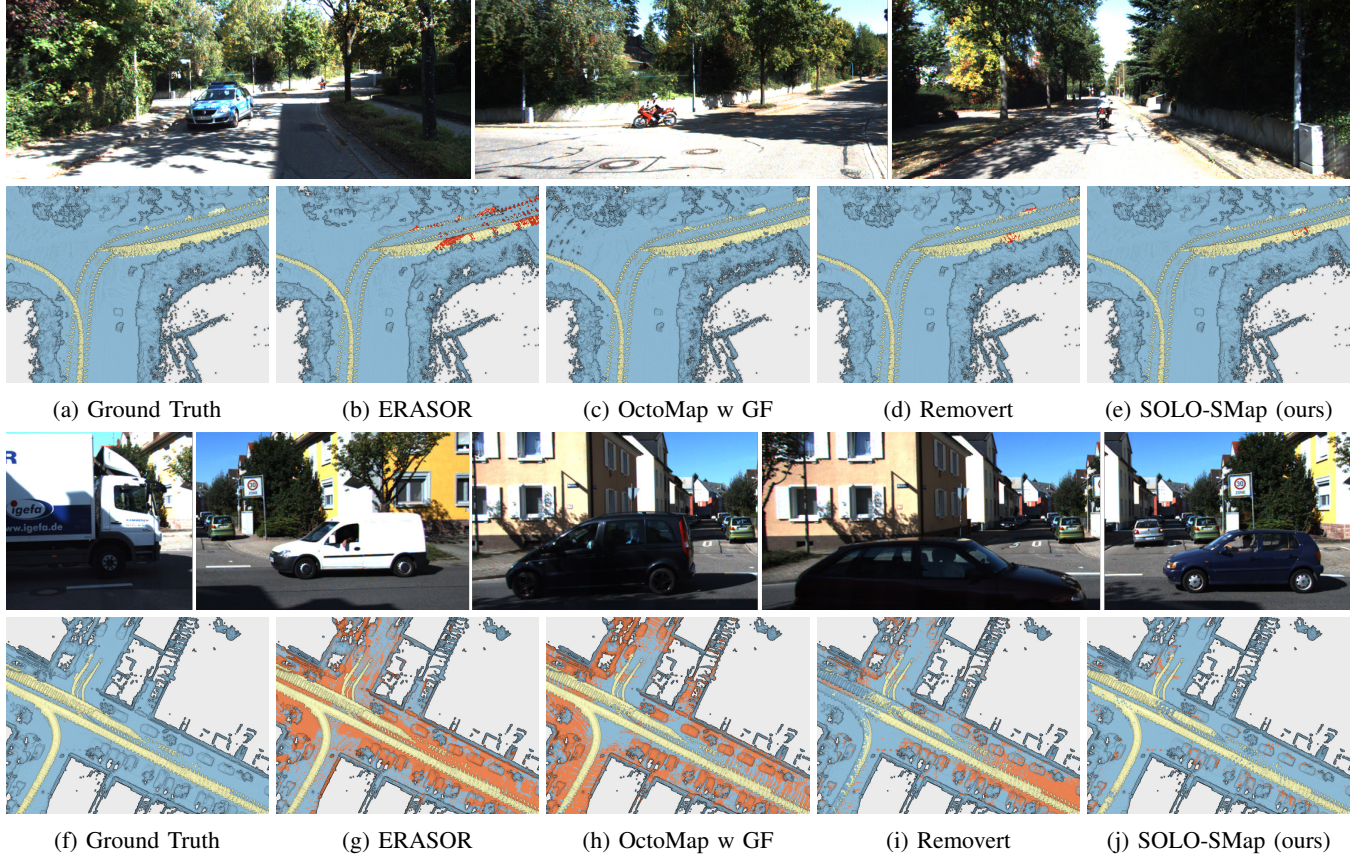


Fig. 7: Qualitative results on sequence 02 (a~e) and 07 (f~j) of SemanticKITTI dataset. The points colored yellow and blue represent true positives and negatives, while orange and red indicate incorrectly classified static and dynamic points.

TABLE II: Quantitative analysis of the influence of pose estimation by different SLAM algorithms for dynamic points removal.

Methods	GT poses of KITTI				SuMa++				SOLO			
	SA	DA	AA	HA	SA	DA	AA	HA	SA	DA	AA	HA
ERASOR	47.63	99.44	68.82	64.41	72.31	98.71	84.48	83.47	75.72	99.11	86.63	85.85
OctoMap w GF	76.69	94.97	85.34	84.86	93.85	96.98	95.40	95.39	95.64	97.05	96.35	96.34
Removert	99.51	33.20	57.48	49.79	99.83	32.42	56.89	48.94	99.49	33.38	57.63	49.99
SMap (ours)	97.26	83.83	90.30	90.05	99.77	85.98	92.62	92.36	99.50	91.83	95.59	95.51

sequence, followed by SOLO-SMap, while ERASOR shows the poorest performance. Fig. 7(f)~(j) display the results on Seq. 07, where the orange points indicates those static points that were excessively removed. Since our method, SOLO-SMap, distinguishes true dynamic points from potential dynamic points, it maximally retains static reference points, which is crucial for ensuring the accuracy of odometry.

C. Real-time performance evaluation

To evaluate the real-time performance of SOLO-SMap, assessed the average time consumption of each main module for processing one LiDAR scan on a laptop (with an Intel i7-10870H CPU and a GeForce RTX 3060 Ti graphics card), as shown in Tab. III. Our semantic segmentation module performs semantic inference as a separate process in the

TABLE III: Average time consumption of modules for processing one scan.

Module	Average Runtime (ms)
Semantic Inference	48.7
Moving Points Detection	7.1
Clustering and Tracking	5.3
Alignment and Optimization	10.8

front end, outputting point clouds with semantic labels at the sensor's frequency. Subsequently, the dynamic-point-robust odometry can quickly output pose estimation. Typically, the convergence time may increase when encountering complex road scenarios, but overall, the algorithm can ensure a stable output at 10Hz.

V. CONCLUSION

In this paper, we proposed an online odometry and static map construction framework for dynamic scenarios based solely on LiDAR point cloud, which is capable of filtering out instance-level dynamic points during the pre-alignment process. Compared to state-of-the-art approaches, the proposed method SOLO-SMap demonstrated significant advantages in localization and mapping accuracy while maintaining strong robustness across various highly dynamic scenarios. Extensive experiments on both public benchmarks and custom-collected datasets confirmed the operational effectiveness of SOLO-SMap across diverse scenarios.

Future work will focus on refining the semantic segmentation network to enhance both computational efficiency and generalization capability. Furthermore, thanks to the modular design of SOLO-SMap, we can seamlessly integrate the IMU motion information to enhance robustness in challenging scenarios that feature geometrical degradations and fast-motion conditions.

REFERENCES

- [1] J. Wen, X. Zhang, Q. Bi, H. Liu, J. Yuan, and Y. Fang, "G²VD Planner: Efficient Motion Planning With Grid-Based Generalized Voronoi Diagrams," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 3743–3755, 2025.
- [2] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [3] H. Yin, S. Li, Y. Tao, J. Guo, and B. Huang, "Dynam-SLAM: An Accurate, Robust Stereo Visual-Inertial SLAM Method in Dynamic Environments," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 289–308, 2022.
- [4] S. Wu, X. Zhang, S. Zhang, Z. Song, R. Wang, and J. Yuan, "MPOC-SLAM: An RGB-D SLAM System With Motion Probability and Object Category in High Dynamic Environments," *IEEE/ASME Transactions on Mechatronics*, doi: 10.1109/TMECH.2024.3410508.
- [5] Z. Yan, X. Wu, Z. Jian, B. Lan, and X. Wang, "RH-Map: Online Map Construction Framework of Dynamic Object Removal Based on 3D Region-wise Hash Map Structure," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1492–1499, 2024.
- [6] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: In Defense of Point-to-Point ICPSimple, Accurate, and Robust Registration if Done the Right Way," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 1029–1036, 2023.
- [7] T. Fan, B. Shen, H. Chen, W. Zhang, and J. Pan, "DynamicFilter: an Online Dynamic Objects Removal Framework for Highly Dynamic Environments," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 7988–7994.
- [8] Z. Chen, K. Zhang, H. Chen, M. Y. Wang, W. Zhang, and H. Yu, "DORF: A Dynamic Object Removal Framework for Robust Static LiDAR Mapping in Urban Environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 12, pp. 7922–7929, 2023.
- [9] A. Milioto, I. Vizzo, J. Behley, and C. Stachniss, "RangeNet++: Fast and Accurate LiDAR Semantic Segmentation," in *2019 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2019, pp. 4213–4220.
- [10] X. Li, G. Zhang, B. Wang, Y. Hu, and B. Yin, "Center Focusing Network for Real-Time LiDAR Panoptic Segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 13 425–13 434.
- [11] F. Zhang, C. Guan, J. Fang, S. Bai, R. Yang, P. H. Torr, and V. Prisacariu, "Instance Segmentation of LiDAR Point Clouds," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 9448–9455.
- [12] J. Cui, J. Chen, and L. Li, "SAGE-ICP: Semantic Information-Assisted ICP," *arXiv preprint arXiv:2310.07237*, 2023.
- [13] S. Pagad, S. Agarwal, S. Narayanan, K. Rangan, H. Kim, and G. Yalla, "Robust Method for Removing Dynamic Objects from Point Clouds," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 10 765–10 771.
- [14] J. Zhang, S. Singh *et al.*, "LOAM: Lidar Odometry and Mapping in Real-time," in *Robotics: Science and systems*, vol. 2, no. 9. Berkeley, CA, 2014, pp. 1–9.
- [15] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, pp. 189–206, 2013.
- [16] J. Schauer and A. Nüchter, "The Peopleremover-Removing Dynamic Objects From 3-D Point Cloud Data by Traversing a Voxel Occupancy Grid," *IEEE robotics and automation letters*, vol. 3, no. 3, pp. 1679–1686, 2018.
- [17] D. Yoon, T. Tang, and T. Barfoot, "Mapless Online Detection of Dynamic Objects in 3D LiDAR," in *2019 16th Conference on Computer and Robot Vision (CRV)*. IEEE, 2019, pp. 113–120.
- [18] G. Kim and A. Kim, "Remove, then Revert: Static Point cloud Map Construction using Multiresolution Range Images," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 10 758–10 765.
- [19] Q. Zhang, D. Duberg, R. Geng, M. Jia, L. Wang, and P. Jensfelt, "A Dynamic Points Removal Benchmark in Point Cloud Maps," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2023, pp. 608–614.
- [20] P. Ruchti and W. Burgard, "Mapping with Dynamic-Object Probabilities Calculated from Single 3D Range Scans," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 6331–6336.
- [21] C. Yu, Z. Liu, X.-J. Liu, F. Xie, Y. Yang, Q. Wei, and Q. Fei, "DS-SLAM: A Semantic Visual SLAM towards Dynamic Environments," in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2018, pp. 1168–1174.
- [22] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 4758–4765.
- [23] C. Qian, Z. Xiang, Z. Wu, and H. Sun, "RF-LIO: Removal-First Tightly-coupled Lidar Inertial Odometry in High Dynamic Environments," *arXiv preprint arXiv:2206.09463*, 2022.
- [24] P. Pfreundschuh, H. F. Hendriks, V. Reijgwart, R. Dubé, R. Siegwart, and A. Cramariuc, "Dynamic Object Aware LiDAR SLAM based on Automatic Generation of Training Data," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 11 641–11 647.
- [25] X. Chen, A. Milioto, E. Palazzolo, P. Giguere, J. Behley, and C. Stachniss, "SuMa++: Efficient LiDAR-based Semantic SLAM," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 4530–4537.
- [26] S. Thrun, "Probabilistic robotics," *Communications of the ACM*, vol. 45, no. 3, pp. 52–57, 2002.
- [27] H. Wu, Y. Li, W. Xu, F. Kong, and F. Zhang, "Moving event detection from LiDAR point streams," *nature communications*, vol. 15, no. 1, p. 345, 2024.
- [28] D. Duberg, Q. Zhang, M. Jia, and P. Jensfelt, "DUFOMap: Efficient Dynamic Awareness Mapping," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5038–5045, 2024.